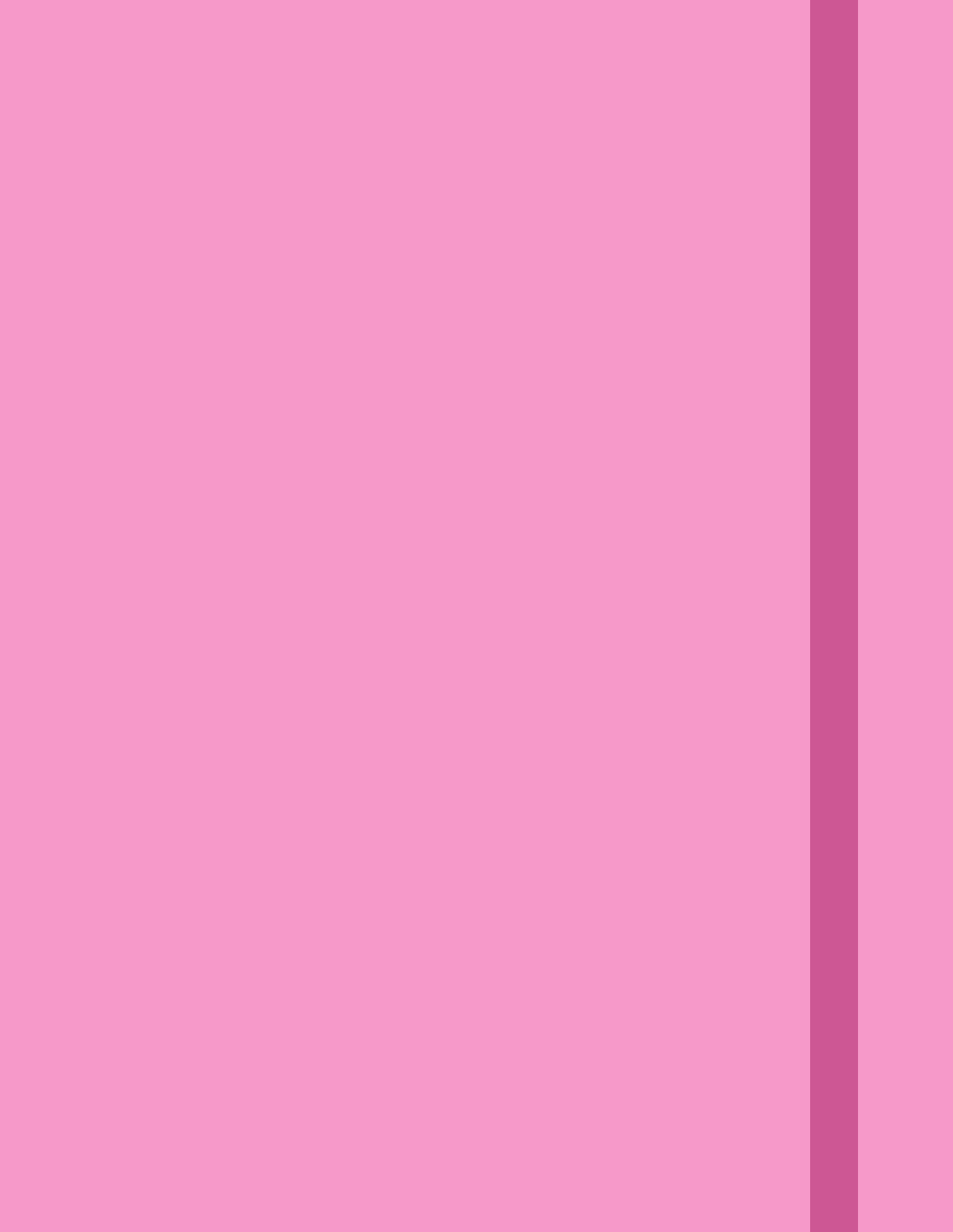




Curse of Dimensionality

Features



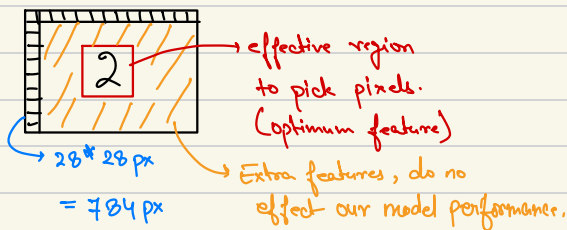
Dimensions.

As features increases there are only few features that are useful.

An increase in feature degrades model's performance and increase computational complexities.

$$\begin{matrix} f_n < f_m \\ (4) & (5) \end{matrix}$$

MNIST Dataset.

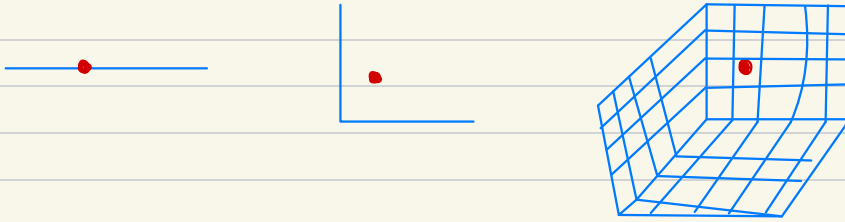


Each pixel is a feature.

In ML we generally have high-dims data.

eg) Images, Text.

Problem : As feature increases it creates sparsity.



Sparsity in the higher dimension.

Curse of Dimensionality :
1. Performance Decreases.
2. Heavy Computation.

Solution :

Dimensionality Reduction

Feature Selection

$f_1, f_2, \dots, f_n$

↓

f_1, f_2, f_{10}

- Forward Selection.
- Backward Elimination.

Feature Extraction.

- PCA
- LDA (Linear Discriminant Analysis)
- T-SNE

PCA | Pt.1 Geometric Intuition

PCA $\rightarrow$ Unsupervised Learning Technique
Very Old Technique.

PCA $\rightarrow$ Principle Component Analysis.

Feature Extraction Technique

Goal : To reduce no. of features.

Introduction :

A photographer takes photo of a scenario (3d)
(2d)

3d $\rightarrow$ 2d
Scene Photo : Reduction of dimension.

PCA is a technique, which can transform a higher dim data to lower dim data while keeping the essence of the data.

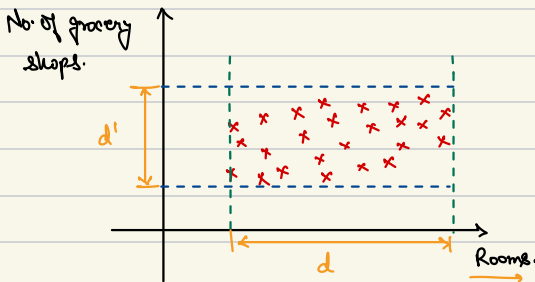
Benefits :

1. Faster execution of algo.
2. Visualization.

Geometric Intuition :

No. of rooms	No. of grocery shops	Price (L)
3	2	60
4	0	130
5	6	170
2	10	90

PCA [Feature Selection] $\rightarrow$ Drop (No. of grocery shops)



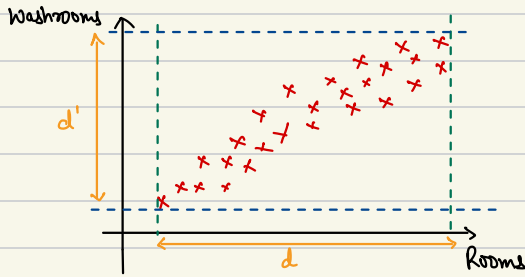
Data is spread more in Rooms column.

$$d > d'$$

Let's say there are few changes in data.

No. of Rooms	No. of washrooms	Price
--------------	------------------	-------

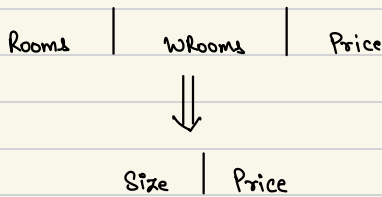
In above case, feature selection is not that important.



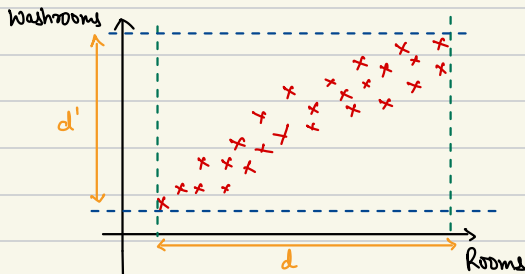
$$d \approx d'$$

Variance is almost same.

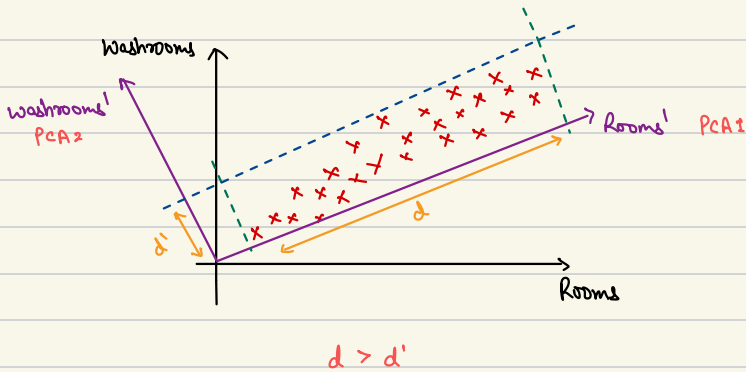
PCA [Feature Extraction]



PCA [Features Extraction] forgets old features and extract information to create new features from the existing features, chooses the set of features from the subset of the newly extracted features.



PCA rotates the graph in a manner that a new axis is formed.



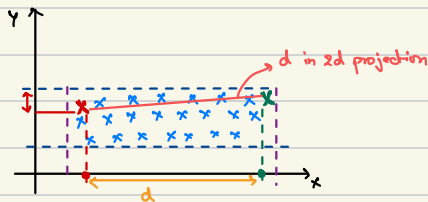
We choose PCA1 as variance is more.
 Transform the data according to PCA1.

Why variance is important?

$$\text{Var} = \sum_{i=1}^n \frac{(x_i - \bar{x})^2}{n}$$

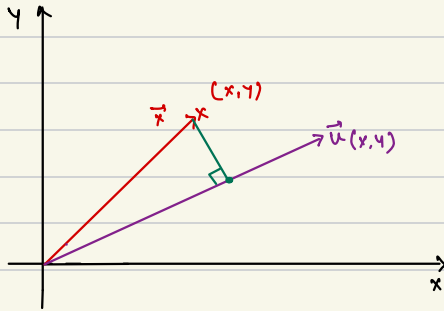
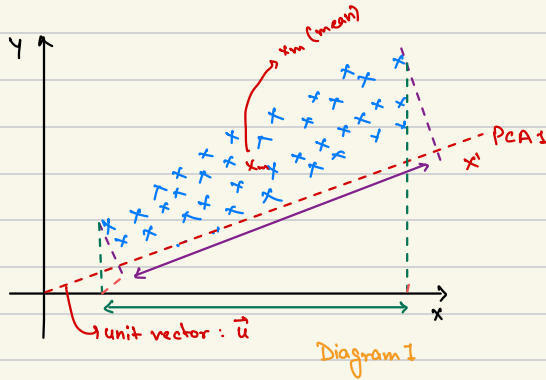
$$\bar{x} = \text{Mean} = \frac{\text{Sum of all } n}{\text{total no. of } n}$$

Spread $\propto$ variance.



If we project data in y axis (2d $\rightarrow$ 1d) then the actual dist. b/w the points would be lost.
 In algo like kNN, dist. is crucial. But when we project it in x-axis (2d $\rightarrow$ 1d) there is still sufficient comparable distance. Thus, variance is important.

PCA Pt.2 | Problem Formulation



$$\text{Projecting } \vec{x} \text{ on } \vec{u} : \frac{\vec{u} \cdot \vec{x}}{|\vec{u}|}$$

$$\text{Here } |\vec{u}| = 1$$

$$\text{Projection : } \vec{u} \cdot \vec{x} \Rightarrow u^T x$$

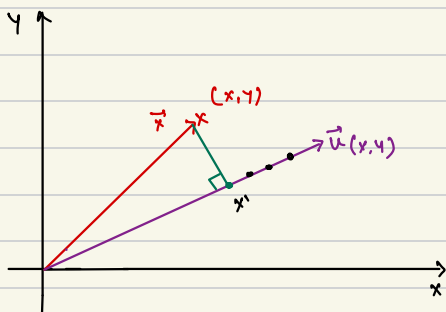
$$\begin{matrix} [x_1 & y_1] & [x_2 & y_2] \\ x & u \end{matrix}$$

$$\begin{bmatrix} x_1 & y_1 \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

$$= x_1 x_2 + y_1 y_2$$

Similarly, we have many points, we project them at u , to get dist.

Maximizing variance.



$$x' = [u^T x]$$

$$[u^T x_1] \quad [u^T x_2] \quad \dots \quad [u^T x_n]$$

Calculating var. for all points.

$$\text{var} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

$$\text{var} = \frac{1}{n} \sum_{i=1}^n (u^T x_i - u^T \bar{x})^2$$

$\bar{x}_m$ is a $\bar{x}$ in the projection
(Diagram 1)

finding the u (unit vector) for which var is maximum.

[Skipping Maths, because it's high level maths]

Co-variance and Co-variance Matrix.:

[Co-variance : See Descriptive Maths]

Co-variance Matrix :

$x_1 | x_2$

Co-var matrix $\rightarrow 2 \times 2$

$$\begin{matrix} x_1 & \begin{bmatrix} \text{Cov}(x_1, x_1) & \text{Cov}(x_1, x_2) \\ \text{Cov}(x_2, x_1) & \text{Cov}(x_2, x_2) \end{bmatrix} \\ x_2 & \end{matrix}$$

$$\text{Cov}(x_1, x_1) = \text{var}(x_1)$$

$$\text{Cov}(x_2, x_2) = \text{var}(x_2)$$

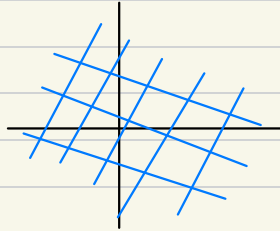
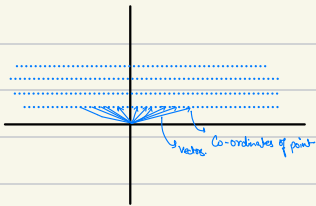
$$\text{Cov}(x_1, x_2) = \text{Cov}(x_2, x_1)$$

$x | y | z$

$$\begin{matrix} x & \begin{bmatrix} V_x & C_{xy} & C_{xz} \\ C_{yx} & V_y & C_{yz} \\ C_{zx} & C_{zy} & V_z \end{bmatrix} \\ y & \\ z & \end{matrix}$$

Linear Transformation, Eigen Vectors and Eigen Values. :

Matrices are linear transformations.



Transformation of Matrix.
(Direction & Magnitude changes)

Eigen vectors $\rightarrow$ Special vectors, when transformation is applied, these vectors does not changes the direction.

Eigen values $\rightarrow$ The factor at which the values of eigen vectors changes is called Eigen Values.

Rule: $A \vec{v} = \lambda \vec{v}$

$A \rightarrow$ Matrix

$\vec{v} \rightarrow$ Eigen vector.

$\lambda \rightarrow$ Eigen value

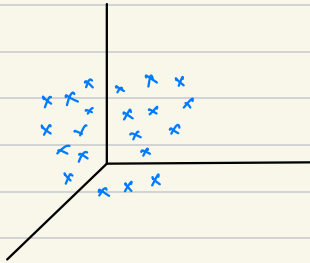
The largest eigen vector for which the eigen value of the co-variance matrix is high gives the higher variance.

$$\text{var} = \frac{1}{n} \sum_{i=1}^n (u^T x_i - u^T \bar{x})^2$$

finding the u (unit vector) for which var is maximum.

Step by Step Solution :

f_1 | f_2 | f_3 | target



Step 1: Find the mean-centering of data.
(for every point subtract by mean)

} improves PCA performance

Step 2: Find the covar matrix.

$$\begin{matrix} f_1 \\ f_2 \\ f_3 \end{matrix} \begin{bmatrix} V_{f_1} & C_{f_1 f_2} & C_{f_1 f_3} \\ C_{f_2 f_1} & V_{f_2} & C_{f_2 f_3} \\ C_{f_3 f_1} & C_{f_3 f_2} & V_{f_3} \end{bmatrix}$$

$f_1 \quad f_2 \quad f_3$

Step 3: Find the eigen value & eigen vector for co-var matrix.

In 3d $\rightarrow$ 3 vectors

3 values $\rightarrow \lambda_1, \lambda_2, \lambda_3$

$\downarrow \quad \quad \downarrow \quad \quad \downarrow$
PCA1 PCA2 PCA3

Either go to 1d or 2d.

How to transform points?

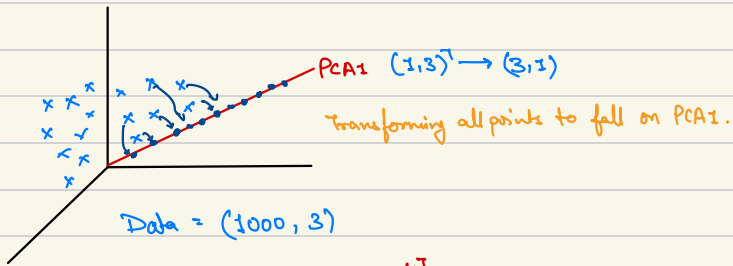
$$3d \rightarrow 2d \mid 1d.$$

$$3d \rightarrow 1d$$

$$\begin{array}{c|c|c|c} f_1 & f_2 & f_3 & \text{Target} \end{array}$$

↓

$$\text{PCA1} \mid \text{Target}.$$



$$u^T \cdot x$$
$$(1000, 3) \cdot \underbrace{(3, 1)}_{u^T} \xrightarrow[\text{product}]{\text{dot product}} (1000, 1)$$

$$3d \rightarrow 2d$$

$$u^T \cdot x$$
$$(1000, 3) \cdot \underbrace{(3, 2)}_{u^T} \longrightarrow (1000, 2)$$

Finding optimum no. of PC :

eigen value = λ

784 $\rightarrow$ $\lambda_1, \lambda_2, \dots, \lambda_{784}$.

$\downarrow$
This EVs tells how much variance of the data is explained by this EVs alone.

Converting EVs to percent.

$$\left(\frac{\lambda_1}{\lambda_1 + \lambda_2 + \dots + \lambda_{784}} \right) \times 100$$

We have to reach till 90%.

$\lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5, \dots, \lambda_{784}$

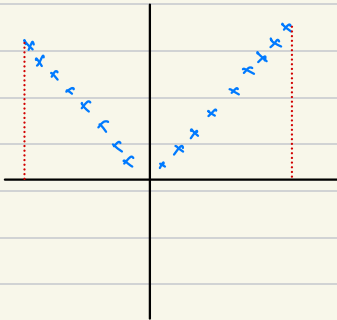
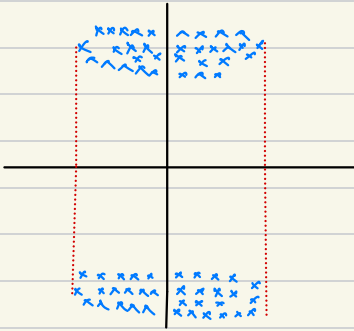
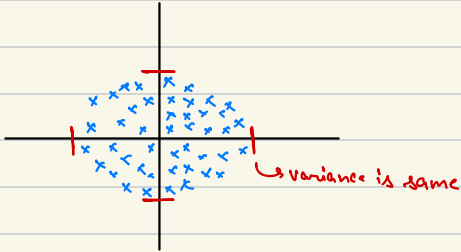
30% 25% 15% 10% 5%

$30 + 25 + 15 + 10 + 5 \rightarrow 85\%$

(Suppose we get 90% till λ_{15})

So, we will take 15 principle components.

When PCA does not work?

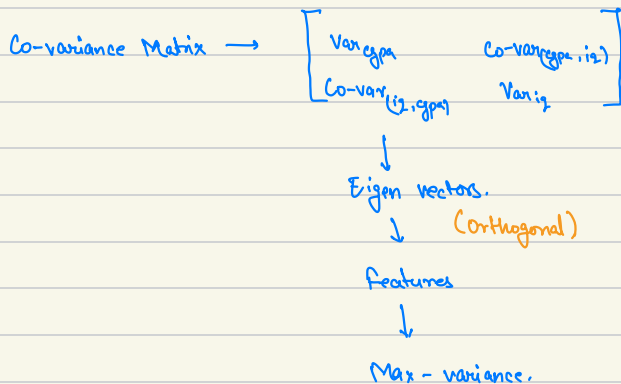


If the data follow a specific pattern (like line curve)

Eigen Vectors in PCA :

$\left[\begin{array}{c|c|c} \text{cgpa} & \text{ig} & \text{lpa} \end{array} \right]$

Feature Extraction.



Eigen Decomposition + PCA Variants

Diagonal Matrix :

$$A = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \quad A = \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix}$$

Diagonal Matrix
stores.

- Power $\rightarrow A^{100} \rightarrow \begin{bmatrix} a^{100} & 0 \\ 0 & b^{100} \end{bmatrix}$

- Eigen value $\rightarrow \lambda(A) = a, b$

- Multiplication by a vector $\rightarrow \begin{bmatrix} x \\ y \end{bmatrix}$ component scale by 'a' factor
component scale by 'b' factor.

- Matrix Multiplication $\rightarrow \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} c & 0 \\ 0 & d \end{bmatrix} \rightarrow \begin{bmatrix} ac & 0 \\ 0 & bd \end{bmatrix}$

Orthogonal Matrix :

It's a sq. matrix, whose columns & rows are orthogonal unit vectors (i.e., orthonormal vectors), meaning that they are all of unit length and are at right angles to each other.

$$\begin{bmatrix} \vec{v}_1 & \vec{v}_2 \\ a & b \\ c & d \end{bmatrix} \rightarrow \text{orthogonal}$$

- Properties:
 $\vec{v}_1 \perp \vec{v}_2$
 $\angle 90^\circ$
i.e. $ab + cd = 0$

$$\sqrt{a^2 + c^2} = 1$$

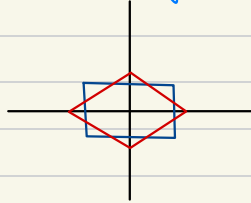
$$\sqrt{b^2 + d^2} = 1$$

$I \in$ Orthogonal Matrix.

Inverse of Orthogonal Matrix = Transpose of Orthogonal matrix.

$$A^{-1} = A^T$$

Perfect rotation, no scaling and shearing.



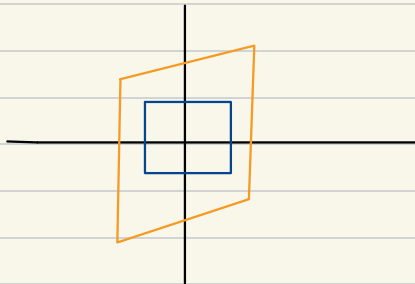
Symmetric Matrix :

Square Matrix

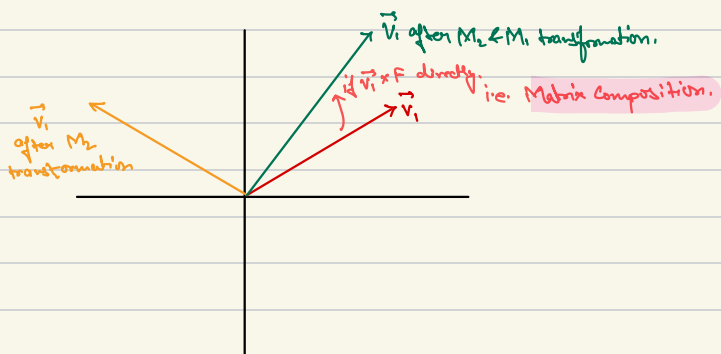
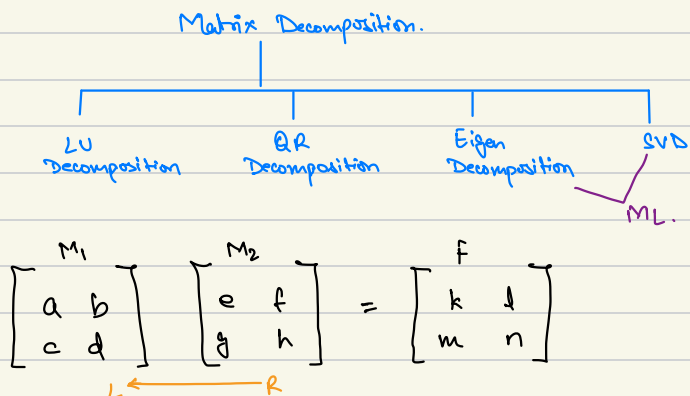
$$A = A^T$$

eg) $A = \begin{bmatrix} a & c \\ c & b \end{bmatrix} \quad A^T = \begin{bmatrix} a & c \\ c & b \end{bmatrix} \Rightarrow \text{Symmetric matrix.}$

- Real Eigen Values.
- Orthogonal Eigenvectors.



Matrix composition :-



vice versa of Matrix composition is Matrix Decomposition.

Eigen Decomposition :-

The eigen decomposition of a matrix A is given by the equation:

$$A = V \Lambda V^{-1}$$

- V is a matrix whose columns are eigenvectors of A .
- Λ is a diagonal matrix whose entries are eigenvalues of A .
- V^{-1} is inverse of V .

Assuming :

- Square Matrix.
- Diagonalizability : for a $n \times n$ matrix it should have a linearly independent eigen vectors.
↳ different directions eigen vectors. 

$$\text{eg:)} \quad \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \lambda_1 \quad \lambda_2 \quad \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \quad \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix}^{-1}$$

Eigen vector : $A \vec{v} = \lambda \vec{v}$

$$A \vec{v}_1 = \lambda_1 \vec{v}_1$$

$$A \vec{v}_2 = \lambda_2 \vec{v}_2$$

$$A V = V \Lambda$$

$$V = [\vec{v}_1 \quad \vec{v}_2]$$

$$\vec{v}_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \quad \vec{v}_2 = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

$$A \vec{v}_1 = \lambda_1 \vec{v}_1$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}_{2 \times 2} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}_{2 \times 1} = \lambda_1 \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

$$\begin{bmatrix} ax_1 + by_1 \\ cx_1 + dy_1 \end{bmatrix} = \lambda_1 \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

$$ax_1 + by_1 = \lambda_1 x_1 \quad \text{--- (1)}$$

$$cx_1 + dy_1 = \lambda_1 y_1 \quad \text{--- (2)}$$

$$A \vec{v}_2 = \lambda_2 \vec{v}_2$$

$$ax_2 + by_2 = \lambda_2 x_2 \quad \text{--- (3)}$$

$$cx_2 + dy_2 = \lambda_2 y_2 \quad \text{--- (4)}$$

$$AV = \Lambda V$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

$$\begin{bmatrix} ax_1 + bx_2 & ax_2 + by_2 \\ cx_1 + dy_1 & cx_2 + dy_2 \end{bmatrix} = \begin{bmatrix} \lambda_1 x_1 & \lambda_2 x_2 \\ \lambda_1 y_1 & \lambda_2 y_2 \end{bmatrix}$$

$$ax_1 + bx_2 = \lambda_1 x_1$$

$$ax_2 + by_2 = \lambda_2 x_2$$

$$cx_1 + dy_1 = \lambda_1 y_1$$

$$cx_2 + dy_2 = \lambda_2 y_2$$

$$\begin{aligned}
 A \vec{v}_1 &= \lambda_1 \vec{v}_1 & \longrightarrow & \quad \quad \quad A \vec{v} = \underbrace{V \Lambda}_{\times V^{-1} \times V^{-1}} \longrightarrow \boxed{A = V \Lambda V^{-1}} \\
 A \vec{v}_2 &= \lambda_2 \vec{v}_2 & & & & \quad \quad \quad \begin{array}{l} \text{eigen} \\ \text{vectors} \end{array} \quad \begin{array}{l} \text{diagonal} \\ \text{eigen} \\ \text{values} \end{array} \quad \text{inverse } V
 \end{aligned}$$

Eigen Decomposition of Symmetric Matrix :-

$$\boxed{A = V \Lambda V^{-1}} \quad \text{spectral decomposition.}$$

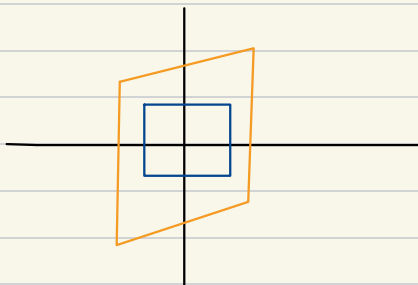
$V \rightarrow$ Orthogonal

$\Lambda \rightarrow$ Diagonal matrix.

Symmetric matrix $\xrightarrow{\text{Decompose}}$ Orthogonal matrix and diagonal matrix.

$$A = V \Lambda V^{-1}$$

$\downarrow$ rotation $\downarrow$ scaling $\downarrow$ rotation.



Advantages of Eigen Decomposition :-

ML (PCA)

Physics

Signal Processing.

Quantum Physics.

$$A_{n \times n} = A^{1000}$$

$$\hookrightarrow A = V \Lambda V^{-1}$$

$$A^{100} = \underbrace{V \Lambda V^{-1}} \cdot \underbrace{V \Lambda V^{-1}} \cdot \underbrace{V \Lambda V^{-1}} \dots$$

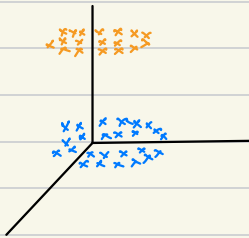
$$A^{100} = V \Lambda^{100} V^{-1}$$

$$A^{100} = V \begin{bmatrix} \lambda_1^{100} & 0 \\ 0 & \lambda_2^{100} \end{bmatrix} V^{-1}.$$

Kernel PCA :-

In non-linear datasets PCA fails
Kernel PCA comes into the picture.

Kernel PCA : $2d \rightarrow 3d \rightarrow 1d$.
 $y = e^{-\gamma x^2} \rightarrow$ 



Kernel PCA is a stochastic in nature.

Kernel SVM is slightly different from Kernel PCA.

Rbf Kernel : $y = e^{-\gamma x^2}$

Note: (Look for coding part)

Singular Value Decomposition

PCA

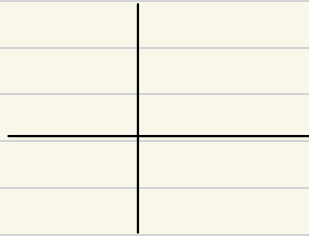
- Eigen value.
- Eigen Decomposition $\rightarrow$ Matrix Decomposition / Factorization.

$\downarrow$
SVD $\rightarrow$ important
+ Difficult.

Non-Square Matrix :

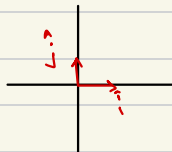
$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}_{2 \times 2} \rightarrow \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3}$$

Linear Transformations of Non-Square matrix. :



$$\begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 3 & 1 \end{bmatrix}_{3 \times 2}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 1 \end{bmatrix}_{2 \times 3}$$



$$\begin{aligned} (x,y) &\rightarrow (x,2,y) \\ (0,1) &\rightarrow (0,0,1) \\ 2d &\rightarrow 3d \end{aligned}$$

$3 \times 2 \rightarrow 3d$
 (Output space) (input space)

$m \times n$
 output space input space

Rectangular Diagonal Matrix :

A matrix that would be diagonal if it were square, but instead rectangular due to extra rows or columns of zeros.

eg: $\begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \end{bmatrix}$ $\begin{bmatrix} a & 0 \\ 0 & b \\ 0 & 0 \end{bmatrix}$

It applies 2 transformations at a time.

$\begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \end{bmatrix} \rightarrow \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$
 (2) (1)

Diagonal matrix
 $\Rightarrow$ scaling

Non-sq. matrix
 $\Rightarrow 3d \rightarrow 2d$

$\begin{bmatrix} a & 0 \\ 0 & b \\ 0 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}_{2 \times 2}$
 3×2 3×2 2×2
 off space. 3×2 2×2

What is SVD?

→ Singular Value Decomposition.

Decomposes the matrix to 3 other matrices.

$$A = U \Sigma V^T$$

Uses :-

1. ML → PCA (Scikit learn)
2. Recomm. Systems.
3. NLP → Latent Semantic Analysis (LSA)
4. CV → image compression.
5. Signal Processing. → Removing Noise.
6. Numerical Linear Algebra
7. Psychometrics. → Scoring.
8. Bioinformatics → gene expression data.
9. Quantum Computing → quantum state tomography to understand state of a quantum system.

SVD The Equation :

$$A = U \Sigma V^T$$

Diagram illustrating the components of the SVD equation:

- A : Finite Matrix
- U : Orthogonal Matrix.
- Σ : Rectangular Diagonal Matrix
- V^T : Orthogonal Matrix.

$$A = U \Sigma V^T$$

$\downarrow$ $m \times n$ $\downarrow$ $m \times m$ $\downarrow$ $n \times n$
 $m \times n$

Eigen Decomposition :

$$A = V \Lambda V^{-1}$$

$\downarrow$ *Symm.* $\downarrow$ *Diagonal*
 $\downarrow$ *Orthogonal*

$$A = U \Sigma V^T$$

$\downarrow$
(non-sq.)

To make non-sq. matrix to sq. matrix : $A A^T$

eg) $A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}_{2 \times 3}$ $A^T = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 1 & 1 \end{bmatrix}_{3 \times 2}$

$$A \cdot A^T = \begin{bmatrix} 6 & 1 \\ 1 & 1 \end{bmatrix}_{2 \times 2}$$

Let $A = U \Sigma V^T$

$$A^T = (U \Sigma V^T)^T$$

$$A \cdot A^T = U \Sigma V^T (U \Sigma V^T)^T$$

$$= U \Sigma \underbrace{V^T V}_{\uparrow} \Sigma^T U^T$$

↑
orthogonal matrix
 $V \cdot V^T = I$
 $V^T = V^{-1}$

$$A \cdot A^T = U \Sigma \underbrace{V^T V}_{I} \Sigma^T U^T$$

$$= U \Sigma \Sigma^T U^T$$

$$A \cdot A^T = U X U^T$$

where $X = \Sigma \Sigma^T$

Eq. 1.

Now

$$A^T \cdot A = (U \Sigma V^T)^T U \Sigma V^T$$

$$= V \Sigma^T \underbrace{U^T U}_{I} \Sigma V^T$$

$$= V \Sigma^T \Sigma V^T$$

$$A^T \cdot A = V Y V^T$$

where $Y = \Sigma^T \Sigma$

Eq. 2.

$$\left. \begin{array}{l} \sum AA^T = U X U^T \rightarrow X = \Sigma \Sigma^T \\ \sum A^T A = V Y V^T \rightarrow Y = \Sigma^T \Sigma \end{array} \right\} \text{Representing Eigen Decomposition.}$$

Symmetric

$U \rightarrow$ Matrix, columns contains eigenvectors of AA^T

$V \rightarrow$ Matrix, columns contains EV of $A^T A$

$$U^T = U^{-1}$$

orthogonal matrix

Basically $A \rightarrow U$ (singular vector) in respect of A : Left Singular Vector.
 $A \rightarrow V$ (singular vector) matrix in respect of A : Right Singular Vector.

Solving Σ ,

$$\begin{cases} x = \Sigma \Sigma^T \\ y = \Sigma^T \Sigma \end{cases}$$

↳ Eigen Values.

$$\begin{matrix} A = U \Sigma V^T \\ (2 \times 3) \quad (2 \times 2) \quad \downarrow \quad (3 \times 3) \\ \quad \quad \quad (2 \times 3) \end{matrix}$$

$$\Sigma = \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \end{bmatrix}$$

$$x = \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \end{bmatrix} \begin{bmatrix} a & 0 \\ 0 & b \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} a^2 & 0 \\ 0 & b^2 \end{bmatrix} \xrightarrow{\lambda} a^2, b^2$$

$$y = \begin{bmatrix} a & 0 \\ 0 & b \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \end{bmatrix} = \begin{bmatrix} a^2 & 0 & 0 \\ 0 & b^2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

↓
λ

$a^2, b^2, 0$

$\sqrt{a^2}, \sqrt{b^2}$

$a, b \rightarrow$ in SVD we call it singular value.
↳ sqrt (eigen values)

$$A = U \Sigma V^T$$

$\uparrow$ left singular value $(A A^T)$
 $\uparrow$ (a, b)
 $\uparrow$ right singular value $(A^T A)$

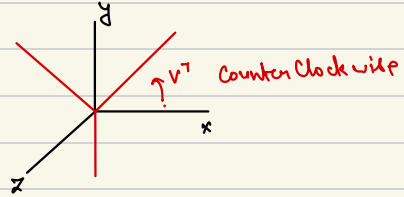
Geometric Intuition :

$$A = U \Sigma V^T$$

$\uparrow$ Matrix Non-sq. (2×3)
 o/p $\uparrow$ 2d space

$$V^T$$

$\uparrow$ eigenvector
 $V = A^T A$
 $\downarrow$ Orthogonal (2d)
 $\downarrow$ 3d
 (3×3)



Σ

$$\Sigma = \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \end{bmatrix}$$

$$\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Σ_2 Σ_1

Scaling

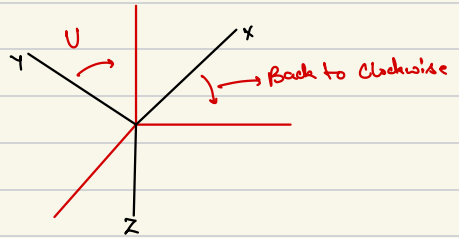
Reduce dim
3d $\rightarrow$ 2d.

$\leftarrow$ R

Stretching.

U

$U \rightarrow EV(AA^T)$
Orthogonal
2d.



Note: Look for Demo 1 and Demo 2 in notes.

How to calculate SVD?

$$A = U \Sigma V^T$$

Simplest technique:

Step 1: AA^T

Step 2: Eigen Decomposition $(AA^T) = U \Lambda U^T$

Step 3: $(\Lambda)^{1/2} \rightarrow \Sigma$

Step 4: $A^T A \rightarrow V$

Step 5: V^T

Numerically unstable.

Multiple techniques are used in SVD internally.

`np.linalg.svd(A)`

$\hookrightarrow U, \Sigma, V^T$
O/p

SVD in PCA :

To find principal components in PCA, there are 2 ways :

1.) Eigen Decomposition.

$$n=1500 \left\{ \begin{array}{c|c|c} \text{Sepal} & \text{Petal} & \text{Petal} \\ \hline - & - & - \\ - & - & - \\ - & - & - \end{array} \right. \rightarrow \text{Co-var Matrix} \rightarrow \text{E-Decomp.}$$

2.) SVD (faster method for larger dataset)

SVD is used in sklearn.

iris_dataset
se | sw | pl | pw

$x \rightarrow (150, 4)$

Co-var $\rightarrow (4, 4)$

*

We directly get $U \Sigma V^T$
without the need of
co-var matrix.

Co-var(x) $\rightarrow$ 1.) Mean Center all col.
2.) $(x^T x) / (n-1) = \text{Covar}$

$$\text{Co-var}(x) = \frac{x_c^T \cdot x_c}{n-1}$$

$$\text{SVD}(x_c) \rightarrow U \Sigma V^T$$

$$x_c \cdot x_c^T = V Y V^T \quad \text{— eigen decomp.}$$

$\hookrightarrow$ Eigenvector of $x^T x$
 $=$ Covar matrix